```
      </div>
    </DataContext.Provider>
  );
};
const ChildA: React.FC = () => {
  return (
    <div>
      <h2>Child A Component</h2>
      <ChildB />
    </div>
  );
};
const ChildB: React.FC = () => {
  return (
    <div>
      <h3>Child B Component</h3>
      <ChildC />
    </div>
  );
};
const ChildC: React.FC = () => {
  const data = useContext(DataContext); #B
  return (
    <div>
      <h4>Child C Component</h4>
      <p>{data.data}</p> #C
    </div>
  );
};
```

This example is located in `use-cases/ContextApiExample.tsx`, and can be executed with the command: `VITE_NAME=ContextApiExample npm run dev`.

Leveraging the Context API eliminates prop drilling, simplifying code maintenance and state management across deeply nested components. However, a significant drawback is the potential for performance bottlenecks, as context changes trigger re-renders in all consuming components regardless of whether they use the updated data, and context-heavy code often suffers from decreased reusability and cumbersome testing.

Probing question (advanced): "In React development, there is frequent discussion around 'simple' vs. 'complex' state management. How would you define the boundary between these two, and what factors influence the decision to adopt a dedicated state management library like Redux or Zustand